

# C++ STL Cheat Sheet

Standard Template Library — Quick Reference

## CONTAINERS

### CONTAINER **vector**

|                                    |                                |
|------------------------------------|--------------------------------|
| <b>v.push_back(x)</b>              | Append x to end                |
| <b>v.pop_back()</b>                | Remove last element            |
| <b>v.insert(it, x)</b>             | Insert x at iterator position  |
| <b>v.erase(it)</b>                 | Remove element at iterator     |
| <b>v.erase(it1, it2)</b>           | Remove range [it1, it2)        |
| <b>v.size()</b> / <b>v.empty()</b> | Size / empty check             |
| <b>v.clear()</b>                   | Remove all elements            |
| <b>v.front()</b> / <b>v.back()</b> | First / last element           |
| <b>v.begin()</b> / <b>v.end()</b>  | Iterators to start / end       |
| <b>v.resize(n)</b>                 | Resize to n elements           |
| <b>v.reserve(n)</b>                | Reserve capacity for n         |
| <b>v[i]</b> / <b>v.at(i)</b>       | Access / bounds-checked access |

### CONTAINER **list**

|                                            |                               |
|--------------------------------------------|-------------------------------|
| <b>l.push_back(x)</b>                      | Append to end                 |
| <b>l.push_front(x)</b>                     | Prepend to front              |
| <b>l.pop_back()</b> / <b>l.pop_front()</b> | Remove last / first           |
| <b>l.insert(it, x)</b>                     | Insert before iterator        |
| <b>l.erase(it)</b>                         | Erase at iterator             |
| <b>l.size()</b> / <b>l.empty()</b>         | Size / empty check            |
| <b>l.front()</b> / <b>l.back()</b>         | First / last element          |
| <b>l.sort()</b>                            | Sort the list                 |
| <b>l.reverse()</b>                         | Reverse the list              |
| <b>l.remove(x)</b>                         | Remove all elements == x      |
| <b>l.unique()</b>                          | Remove consecutive duplicates |
| <b>l.merge(l2)</b>                         | Merge sorted list l2 into l   |

### CONTAINER **stack**

|                                    |                    |
|------------------------------------|--------------------|
| <b>s.push(x)</b>                   | Push x on top      |
| <b>s.pop()</b>                     | Remove top element |
| <b>s.top()</b>                     | Access top element |
| <b>s.size()</b> / <b>s.empty()</b> | Size / empty check |

### CONTAINER **queue**

|                                    |                       |
|------------------------------------|-----------------------|
| <b>q.push(x)</b>                   | Enqueue x at back     |
| <b>q.pop()</b>                     | Dequeue front element |
| <b>q.front()</b>                   | Access front element  |
| <b>q.back()</b>                    | Access back element   |
| <b>q.size()</b> / <b>q.empty()</b> | Size / empty check    |

### CONTAINER **priority\_queue**

|                                                  |                          |
|--------------------------------------------------|--------------------------|
| <b>pq.push(x)</b>                                | Insert x                 |
| <b>pq.pop()</b>                                  | Remove top (max) element |
| <b>pq.top()</b>                                  | Access top (max) element |
| <b>pq.size()</b> / <b>pq.empty()</b>             | Size / empty check       |
| <b>priority_queue,</b><br><b>greater&lt; pq;</b> | Min-heap declaration     |

### CONTAINER **map**

|                                    |                           |
|------------------------------------|---------------------------|
| <b>m[key] = val</b>                | Insert or update key      |
| <b>m.insert({key, val})</b>        | Insert pair               |
| <b>m.erase(key)</b>                | Remove by key             |
| <b>m.find(key)</b>                 | Iterator to key, or end() |
| <b>m.count(key)</b>                | 1 if key exists, else 0   |
| <b>m.size()</b> / <b>m.empty()</b> | Size / empty check        |
| <b>m.clear()</b>                   | Remove all entries        |
| <b>m.lower_bound(key)</b>          | First key >= given key    |
| <b>m.upper_bound(key)</b>          | First key > given key     |
| <b>for(auto&amp; [k,v] : m)</b>    | Range-based iteration     |

### CONTAINER **unordered\_map**

|                                      |                           |
|--------------------------------------|---------------------------|
| <b>um[key] = val</b>                 | Insert or update          |
| <b>um.insert({key, val})</b>         | Insert pair               |
| <b>um.erase(key)</b>                 | Remove by key             |
| <b>um.find(key)</b>                  | Iterator to key, or end() |
| <b>um.count(key)</b>                 | 1 if key exists, else 0   |
| <b>um.size()</b> / <b>um.empty()</b> | Size / empty check        |
| <b>um.clear()</b>                    | Remove all entries        |
| <b>for(auto&amp; [k,v] : um)</b>     | Iterate (unordered)       |

### CONTAINER **multimap**

|                              |                                   |
|------------------------------|-----------------------------------|
| <b>mm.insert({key, val})</b> | Insert (duplicate keys ok)        |
| <b>mm.erase(key)</b>         | Remove all entries with key       |
| <b>mm.find(key)</b>          | Iterator to first match           |
| <b>mm.count(key)</b>         | Count of entries with key         |
| <b>mm.equal_range(key)</b>   | Pair of iterators for all matches |
| <b>mm.lower_bound(key)</b>   | First key >= given key            |
| <b>mm.upper_bound(key)</b>   | First key > given key             |

**CONTAINER** **set**

|                                    |                              |
|------------------------------------|------------------------------|
| <b>s.insert(x)</b>                 | Insert x (ignored if exists) |
| <b>s.erase(x)</b>                  | Remove x                     |
| <b>s.find(x)</b>                   | Iterator to x, or end()      |
| <b>s.count(x)</b>                  | 1 if x exists, else 0        |
| <b>s.size()</b> / <b>s.empty()</b> | Size / empty check           |
| <b>s.clear()</b>                   | Remove all elements          |
| <b>s.lower_bound(x)</b>            | First element $\geq$ x       |
| <b>s.upper_bound(x)</b>            | First element $>$ x          |

**CONTAINER** **unordered\_set**

|                                      |                         |
|--------------------------------------|-------------------------|
| <b>us.insert(x)</b>                  | Insert x                |
| <b>us.erase(x)</b>                   | Remove x                |
| <b>us.find(x)</b>                    | Iterator to x, or end() |
| <b>us.count(x)</b>                   | 1 if x exists, else 0   |
| <b>us.size()</b> / <b>us.empty()</b> | Size / empty check      |
| <b>us.clear()</b>                    | Remove all elements     |

**CONTAINER** **multiset**

|                             |                               |
|-----------------------------|-------------------------------|
| <b>ms.insert(x)</b>         | Insert x (duplicates allowed) |
| <b>ms.erase(ms.find(x))</b> | Remove one occurrence of x    |
| <b>ms.erase(x)</b>          | Remove all occurrences of x   |
| <b>ms.count(x)</b>          | Count of x in set             |
| <b>ms.find(x)</b>           | Iterator to first x           |
| <b>ms.equal_range(x)</b>    | Pair of iterators for all x   |
| <b>ms.lower_bound(x)</b>    | First element $\geq$ x        |
| <b>ms.upper_bound(x)</b>    | First element $>$ x           |

**ALGORITHMS & UTILITIES****UTILITY** **pair**

|                                  |                               |
|----------------------------------|-------------------------------|
| <b>make_pair(a, b)</b>           | Create a pair                 |
| <b>p.first</b> / <b>p.second</b> | Access first / second element |
| <b>p1 &lt; p2</b>                | Lexicographic comparison      |
| <b>tie(a, b) = p</b>             | Unpack pair into variables    |

**ALGORITHM** **sort / reverse**

|                                   |                                     |
|-----------------------------------|-------------------------------------|
| <b>sort(begin, end)</b>           | Sort range ascending                |
| <b>sort(begin, end, cmp)</b>      | Sort with custom comparator         |
| <b>stable_sort(begin, end)</b>    | Sort preserving equal-element order |
| <b>reverse(begin, end)</b>        | Reverse a range in place            |
| <b>sort(v.rbegin(), v.rend())</b> | Sort descending                     |

**ALGORITHM** **find / count / binary\_search**

|                                       |                                  |
|---------------------------------------|----------------------------------|
| <b>find(begin, end, val)</b>          | Iterator to first val, or end    |
| <b>find_if(begin, end, pred)</b>      | First element matching predicate |
| <b>count(begin, end, val)</b>         | Number of occurrences of val     |
| <b>count_if(begin, end, pred)</b>     | Count matching predicate         |
| <b>binary_search(begin, end, val)</b> | True if val found (sorted range) |

**ALGORITHM** **lower\_bound / upper\_bound**

|                                     |                                      |
|-------------------------------------|--------------------------------------|
| <b>lower_bound(begin, end, val)</b> | Iterator to first element $\geq$ val |
| <b>upper_bound(begin, end, val)</b> | Iterator to first element $>$ val    |
| <b>lower_bound - begin</b>          | Index of first $\geq$ val            |
| <b>upper_bound - lower_bound</b>    | Count of elements $==$ val           |

**ALGORITHM** **min / max / swap**

|                                     |                              |
|-------------------------------------|------------------------------|
| <b>min(a, b)</b> / <b>max(a, b)</b> | Smaller / larger of a and b  |
| <b>min(a, b, cmp)</b>               | Smaller by custom comparator |
| <b>min_element(begin, end)</b>      | Iterator to minimum element  |
| <b>max_element(begin, end)</b>      | Iterator to maximum element  |
| <b>swap(a, b)</b>                   | Swap two values in place     |
| <b>minmax(a, b)</b>                 | Pair {min, max}              |
| <b>minmax_element(begin, end)</b>   | Pair of iterators {min, max} |

**ALGORITHM** **next/prev\_permutation**

|                                     |                                      |
|-------------------------------------|--------------------------------------|
| <b>next_permutation(begin, end)</b> | Next permutation, true if exists     |
| <b>prev_permutation(begin, end)</b> | Previous permutation, true if exists |
| <b>// sort first!</b>               | Must be sorted to get all perms      |

